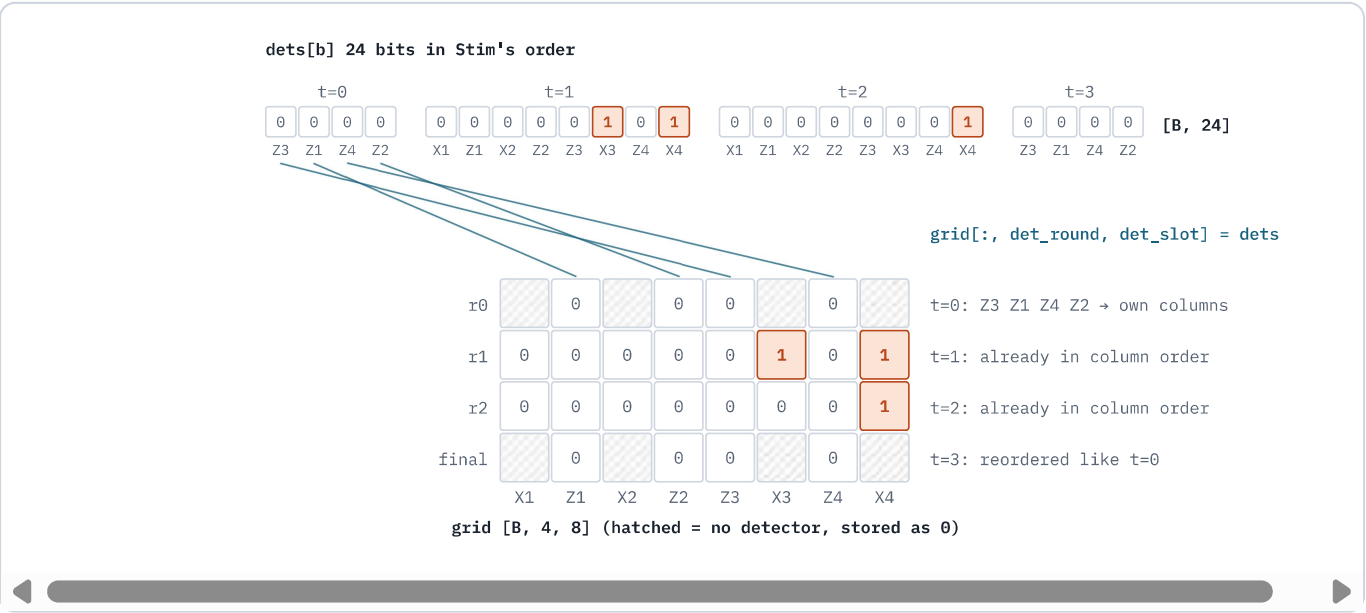
Every picture uses the same real Stim shot: X3 and X4 fire in round 1, X4 fires again in round 2, and the label is 0. Orange marks a detection event; hatching marks a slot with no detector.

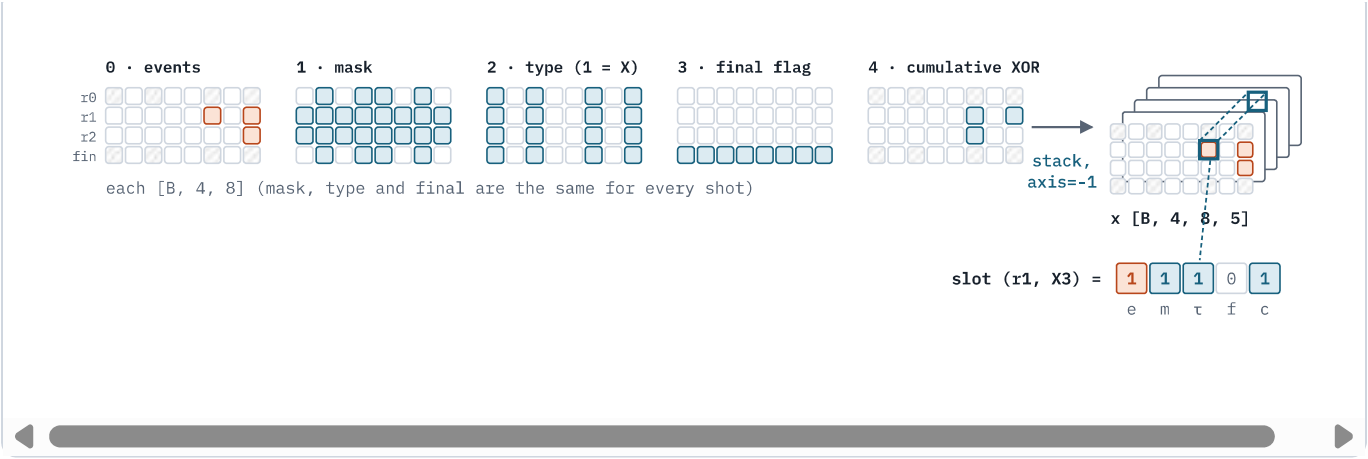
SHARED BY EVERY MODEL

1 · Stim row to [rounds, stabilizer] grid



Stim lists the 24 detectors grouped by time. Each detector's coordinates give its row (round) and column (stabilizer). Rounds 1 and 2 are already in column order; round 0 and the final row are listed Z3 Z1 Z4 Z2, so their bits move.

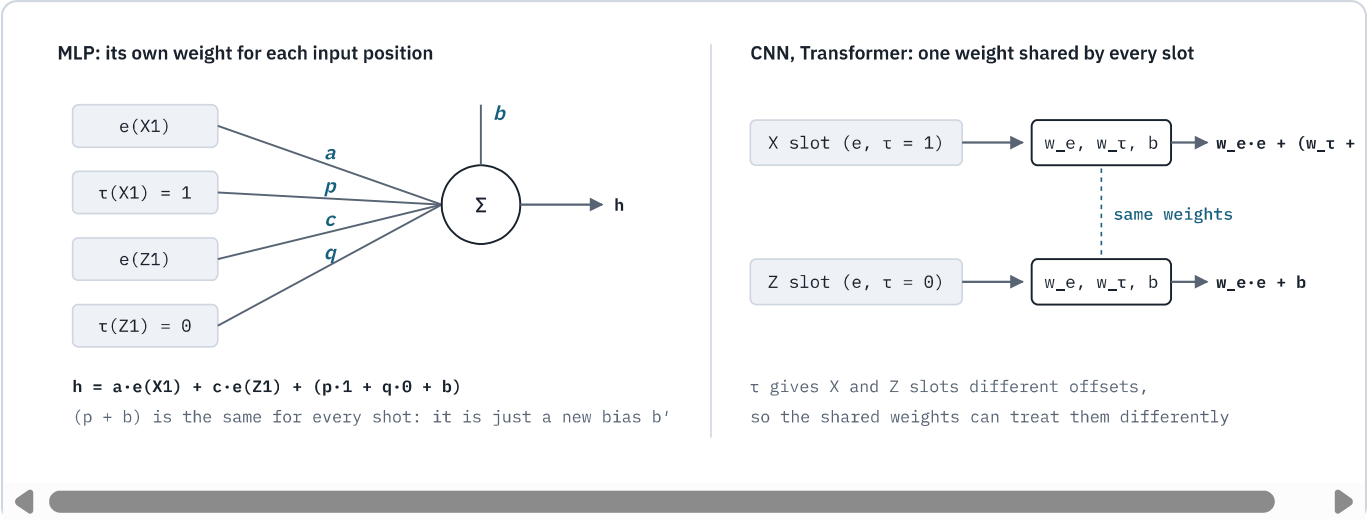
2 · Add channels



Five grids of the same shape are stacked on a new last axis, so every slot holds 5 numbers. Only events and cumulative XOR change from shot to shot; mask, type and final flag come from the circuit and are broadcast, not stored per shot.

> You will also see this written as `[B, C, R, S]`, channels first. It is the same 160 numbers per shot in a different axis order: `x.permute(0, 3, 1, 2)`. Linear, GRU and attention layers read features from the last axis, so they want channels last. Convolution layers want channels right after the batch. We store channels last and permute only for the CNN.

What "folds into the bias" means



Every channel is extra input data: more numbers per slot. A channel only adds information if it differs between the inputs that one weight sees.

In an MLP each input position has its own weight. A channel that is identical in every shot always contributes the same amount, so $p \cdot 1 + q \cdot 0$ is just a fixed number added to every output. The bias b can already learn any fixed number, so the network gains nothing from that input.

When one weight is shared, the same w meets X slots and Z slots, real and empty slots, round 0 and the final row. The constant channels are then the only way that weight can tell those apart, so they carry real information. This holds while all shots come from one circuit; once you mix bases, distances or round counts, the constant channels differ between shots and help every model.

Where each axis goes

A PyTorch layer treats each axis of its input in one of three ways. Which way decides where rounds, stabilizers and channels have to go.

Kind of axis	What the layer does along it	What belongs there
Batch (leading axes)	Applies the same weights to every entry separately. Nothing is mixed across it.	Shots: each is decoded on its own.
Feature (last axis for Linear, GRU and attention; axis 1 for convolution)	Multiplies by a learned matrix. Every entry gets its own weight and all entries are combined.	Channels: the numbers that describe one slot, which should be weighed against each other.
Structure (time for a GRU, positions for a convolution, tokens for attention)	Reuses the same weights at every position and relates positions to each other: in order, by neighbourhood, or by content.	Rounds (time) and stabilizers (space), where the same physics repeats at every position.

A plain Linear layer on our tensor shows the rule. It only multiplies the last axis, so rounds and stabilizers act as batch axes: the same matrix is applied to every slot of every round, and nothing is mixed between slots.

$$x \in \mathbb{R}^{B\times R\times S\times C}, \qquad y_{b,r,s,:} = W\,x_{b,r,s,:} + b, \qquad W \in \mathbb{R}^{H\times C}$$

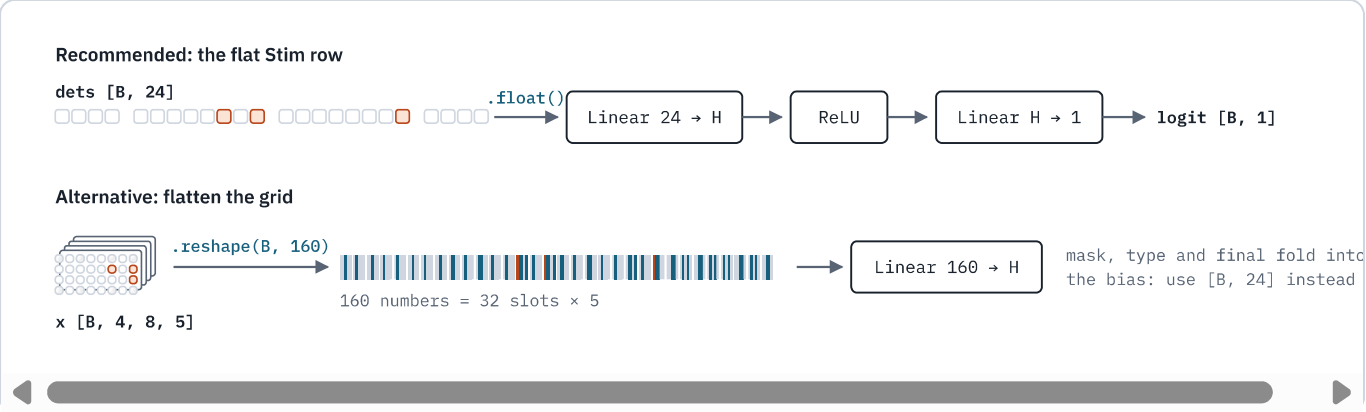
Mixing across rounds and stabilizers is the job of each model's structure layer, and each model puts those two axes somewhere different:

Model	Shots B	Rounds R	Stabilizers S	Channels C
MLP	`axis 0`	flattened together into the feature axis (24 detectors)		not used: events only

RNN	`axis 0`	`axis 1`: time steps	`axis 2`: merged into 40 features	
CNN	`axis 0`	`axis 2`: depth	`axes 3, 4`: height × width on the chip	`axis 1`: channels
Transformer	`axis 0`	`axis 1`: merged into 32 tokens, plus round and slot embeddings		`axis 2`: features
Graph network	node axis, plus a batch vector	a node feature (t)	node features (x, y)	node features

EACH MODEL'S RESHAPE

MLP



`nn.Linear` takes `[N, *, in_features]`. The flat Stim row already has only real detectors, so the MLP needs no grid, no mask and no constant channels. Flattening the full grid works too, but adds 136 inputs that carry nothing new.

WHERE EACH AXIS GOES

Grid axis	Goes to	Why
Shots B	`axis 0, batch`	Each shot is decoded on its own.
Rounds R and stabilizers S	`axis 1`, flattened into 24 features	A Linear layer only combines the last axis, so every detector the MLP must relate has to sit in it.
Channels C	not used	Mask, type and final flag fold into the bias. The events are the only channel that varies.

WHY THIS SHAPE

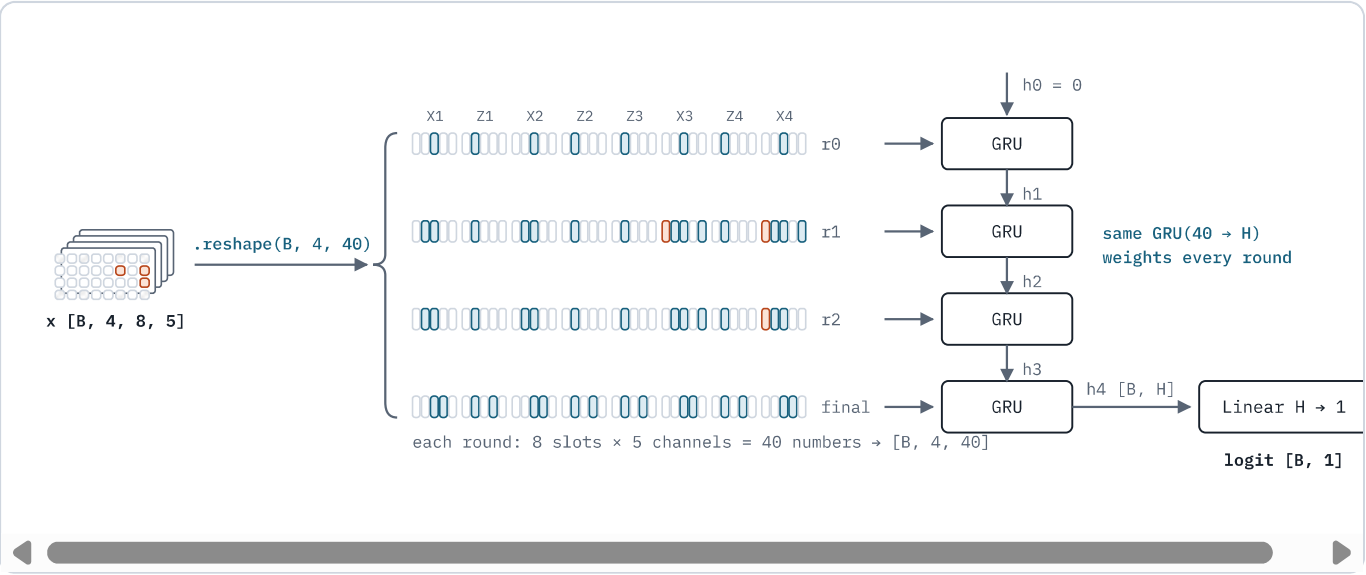
$$h = \mathrm{ReLU}(W_1 v + b_1), \qquad z = w_2^\top h + b_2, \qquad p(\mathrm{flip}) = \sigma(z), \qquad v \in \{0,1\}^{24},\ W_1 \in$$

One hidden unit computes $h_j = \mathrm{ReLU}\big(\sum_{k=1}^{24} W_1[j,k]\, v_k + b_1[j]\big)$. Column k of W_1 belongs to one detector, so the layer sees every round and stabilizer at once. It has no idea which detectors are neighbours: reordering the 24 inputs only reorders the columns of W_1 ($W_1 v = W_1[:,\pi]\, v_\pi$). The order just has to be the same for every shot.

HOW IT LEARNS

Each hidden unit becomes a template for one pattern of detectors, for example "Z4 fires at t = 1 and t = 2", which signals a measurement error. Training lowers the binary cross-entropy between $\sigma(z)$ and the label, adjusting every weight directly. Nothing is shared, so the same kind of error has to be learned separately at every place and round, and the input size is fixed: a new distance or round count needs a new model.

RNN (GRU)



`nn.GRU(batch_first=True)` takes `[N, L, H_in]` : here L = 4 rounds and H_in = 40. Inside one round each slot has its own weights, so the type channel folds into the bias. Mask and final flag differ between rounds, so they tell the shared weights whether they are reading round 0, a middle round or the final row. For shots with different round counts, pad and use `pack_padded_sequence` .

WHERE EACH AXIS GOES

WHERE EACH AXIS GOES

Grid axis	Goes to	Why
Shots B	`axis 0, batch`	Each shot is decoded on its own.
Rounds R	`axis 1`, time steps (L = 4)	The GRU loops over this axis in order with the same weights. Order in time is what matters for rounds.
Stabilizers S and channels C	`axis 2`, merged into 40 features	A GRU has only one feature axis. Index $k = 5s + c$ keeps each slot's 5 numbers together.

Putting stabilizers on the time axis instead would make the GRU read X1, Z1, X2, ... one after another as if they happened in sequence.

WHY THIS SHAPE

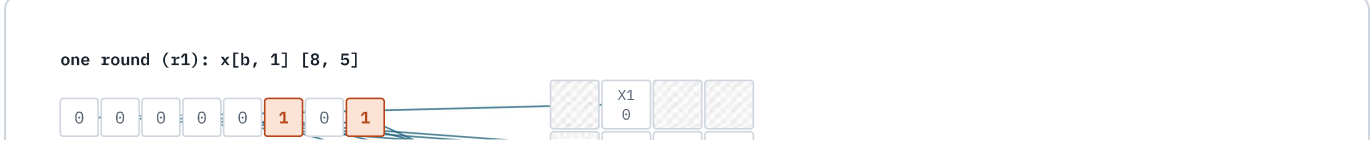
$$
\begin{aligned}
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) && \textbf{reset gate} \\
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) && \textbf{update gate} \\
n_t &= \tanh\big(W_n x_t + b_n + r_t \odot (U_n h_{t-1} + c_n)\big) && \textbf{candidate} \\
h_t &= (1 - z_t) \odot n_t + z_t \odot h_{t-1} && \textbf{new state}
\end{aligned}
$$

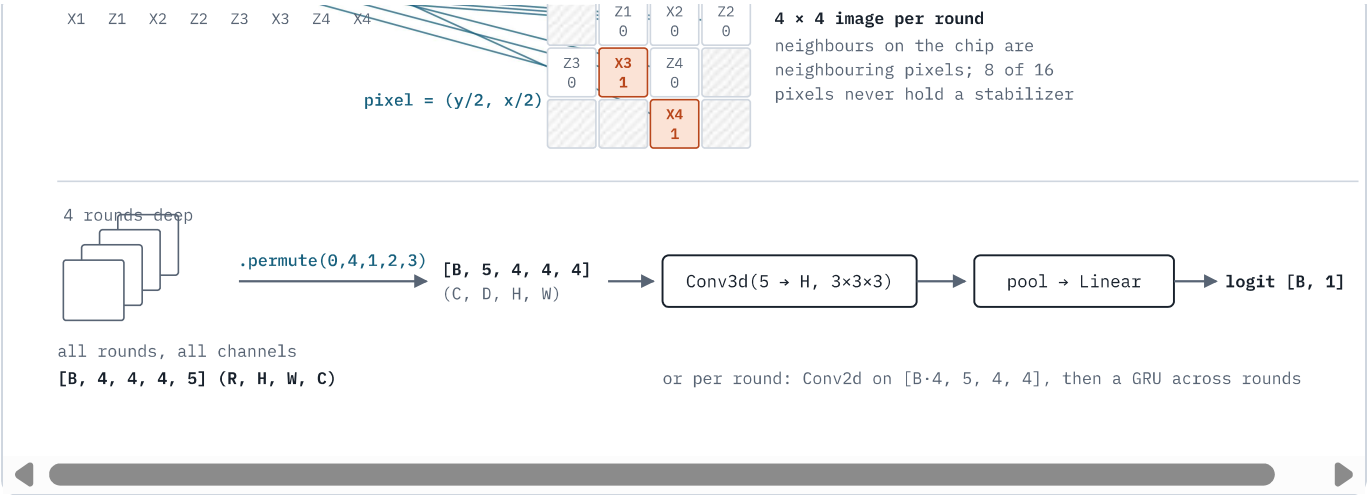
Here $x_t \in \mathbb{R}^{40}$ is round $t = 1, \dots, 4$, $W \in \mathbb{R}^{H \times 40}$, $U \in \mathbb{R}^{H \times H}$ and $h_0 = 0$. The same W and U are used at every t, which is why rounds go on the axis the GRU loops over. Each column of W belongs to one (stabilizer, channel) pair, so inside a round every slot keeps its own weights. The logit is $z = w^\top h_4 + b$.

HOW IT LEARNS

The hidden state h_t is a running summary of everything seen so far, in effect a learned memory of which detection events are still unexplained. Each round, the update gate decides how much of the old memory to keep and the candidate writes in what is new. Training backpropagates through all rounds, and because one W serves every round, its gradient is the sum of each round's contribution. It learns what a single round looks like, so it can run on more rounds than it was trained on.

CNN





`nn.Conv3d` takes `[N, C, D, H, W]`, and `nn.Conv2d` takes `[N, C, H, W]`. The kernel slides over every pixel with the same weights, so it needs all five channels: type to tell X from Z, mask to tell empty pixels from quiet ones, and the final flag for the last round. Conv layers have no mask argument; the mask channel is the only way to give it.

WHERE EACH AXIS GOES

Grid axis	Goes to	Why
Shots B	`axis 0, batch`	Each shot is decoded on its own.
Channels C	`axis 1`, channels	Convolution weighs each channel separately and sums them. PyTorch expects them right after the batch: `[N, C, D, H, W]`.
Rounds R	`axis 2`, depth	The kernel slides along it with shared weights, so a 3-deep kernel always sees 3 consecutive rounds.
Stabilizers S	`axes 3, 4`, height × width	The 8-slot list is unfolded to chip positions (row y/2, column x/2) so that neighbouring stabilizers are neighbouring pixels.

WHY THIS SHAPE

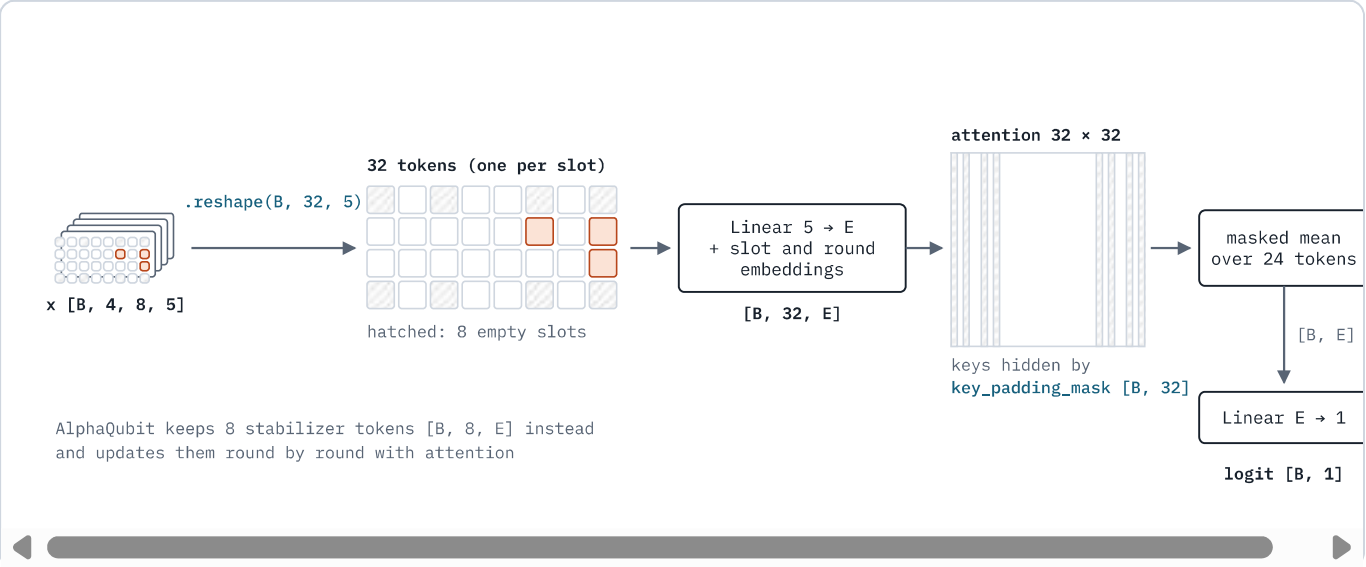
$$y_{b,o,t,i,j} = \beta_o + \sum_{c=1}^{5} \sum_{\Delta t,\, \Delta i,\, \Delta j \in \{-1,0,1\}} K_{o,c,\Delta t,\Delta i,\Delta j}\; x_{b,\, c,\, t+\Delta t,\, i+\Delta i,\, j+\Delta j}, \qquad K \in \mathbb{R}^{H\times 5\times 3\times 3}$$

The sum over c gives each channel its own weight. The sums over $\Delta t, \Delta i, \Delta j$ use the same kernel at every position, and only ever reach one step away. So one step along an axis has to mean one step in the physics: one round along depth, one stabilizer over on the chip along height and width. In the 8-slot list, Z1 and X3 share two data qubits but sit 4 columns apart, which a 3-wide kernel would never connect.

HOW IT LEARNS

Each kernel becomes a detector for a small error signature: two neighbouring checks firing in the same round (a data-qubit error between them), or one check firing in two consecutive rounds (a measurement error). The kernel is shared, so a pattern learned in one place is recognised everywhere. Each extra layer widens what one output can see (3, then 5, then 7 positions per side), so deeper layers join local signatures into longer chains before pooling turns them into one logit. Zero padding at the edges, plus the mask and type channels, tell it where the patch ends and which pixels are empty.

Transformer



`nn.TransformerEncoderLayer(batch_first=True)` takes `[N, L, E]` plus `src_key_padding_mask` `[N, L]` (True = ignore). Attention treats its tokens as an unordered set, so slot identity has to come from the input: the type channel plus a learned embedding for each slot and round.

WHERE EACH AXIS GOES

Grid axis	Goes to	Why
Shots B	`axis 0, batch`	Each shot is decoded on its own.
Rounds R and stabilizers S	`axis 1`, merged into 32 tokens ($\ell = 8r + s$)	Attention relates every token to every other token along this axis.
Channels C	`axis 2`, features (5, embedded to E)	The embedding and the query, key and value matrices all act on the last axis.

The token axis carries no sense of order, so which round and which slot a token is goes into its features as learned embeddings.

WHY THIS SHAPE

$$u_\ell = W_e\, x_\ell + p_{s(\ell)} + q_{r(\ell)}, \qquad U = [u_1, \ldots, u_{32}]^\top \in \mathbb{R}^{32\times E}$$

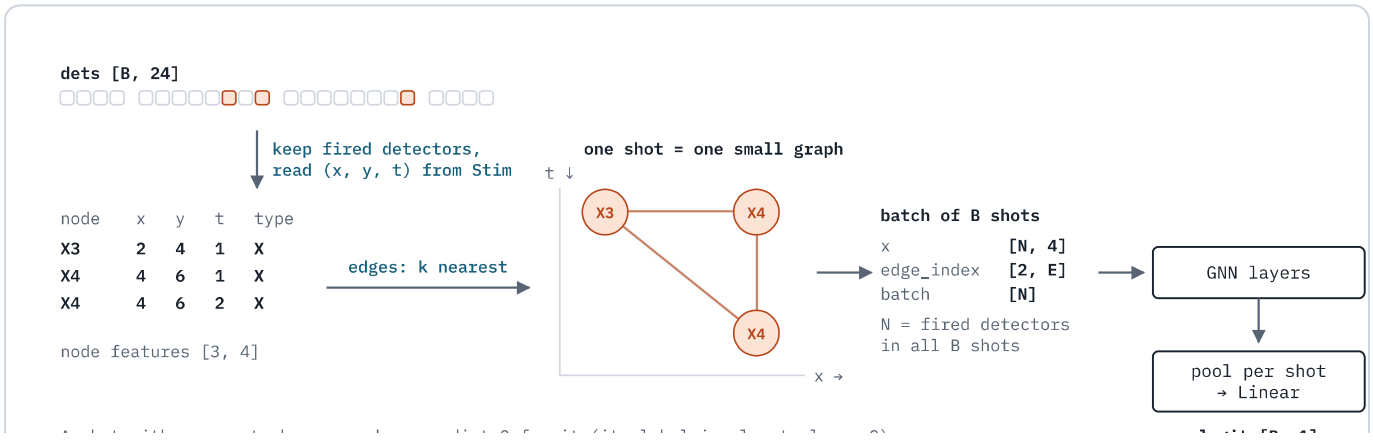
$$A = \mathrm{softmax}\left(\frac{(UW_Q)(UW_K)^\top}{\sqrt{E}} + M\right) \in \mathbb{R}^{32\times 32}, \qquad U' = A\,(UW_V)$$

p_s and q_r are learned vectors for each slot and round, and M puts $-\infty$ on the 8 empty slots. A holds a weight for every pair of tokens, computed from their content rather than fixed by position. Shuffling the tokens with a permutation P just shuffles the output, $\mathrm{Attn}(PU) = P\,\mathrm{Attn}(U)$, so without p and q the layer cannot tell which slot is which. Every learned matrix acts on the feature axis, which is why channels go last.

HOW IT LEARNS

Each token forms a query (what it is looking for) and a key (what it is), and takes in information from the tokens whose keys match its query. A detection event can find its partner, the other end of the same error chain, in a single layer however far apart they are, where a CNN needs several layers. The price is the 32 × 32 table, which grows with the square of the token count: at d = 5 with 25 rounds there are 624 tokens and about 390,000 pairs. That is why AlphaQubit attends over the stabilizers of one round at a time and carries information between rounds with recurrence.

Graph network



```
A shot with no events has no nodes: predict 0 for it (its label is almost always 0).   logit [B, 1]
```

Graph layers (for example in PyTorch Geometric, a separate library) take node features `[N, F]`, an edge list `[2, E]` and a `batch` vector saying which shot each node belongs to. This model skips the grid: only real detectors that fired become nodes, so no mask is needed.

WHERE EACH AXIS GOES

Grid axis	Goes to	Why
Shots B	concatenated on the node axis, plus a `batch` vector	Shots have different numbers of nodes, so they cannot be stacked on a fixed axis.
Rounds R	not an axis: t becomes a node feature	Only rounds where something fired appear at all.
Stabilizers S	not an axis: (x, y) become node features	Geometry is carried as numbers, and edges link events that are close.
Channels C	`axis 1` of the node matrix: (x, y, t, type)	The learned matrices act on this axis.

WHY THIS SHAPE

$$h_i^{(k+1)} = \mathrm{ReLU}\Big(W_1\, h_i^{(k)} + \sum_{j \in \mathcal{N}(i)} w_{ij}\, W_2\, h_j^{(k)}\Big), \qquad g_b = \frac{1}{n_b} \sum_{i\,:\,\mathrm{batch}_i = b} h_i^{(K)}, \qquad z_b = w^\top$$

$h_i^{(0)}$ is node i's feature row, $\mathcal{N}(i)$ its neighbours, and w_{ij} an edge weight, for example one that shrinks with distance. W_1 and W_2 are shared by every node and edge, and the sum over neighbours works for any number of them. So a shot with 3 events and a shot with 30 use the same weights, and the pooled g_b always has the same size whatever the shot looked like.

HOW IT LEARNS

Each layer passes messages along edges, so after k layers a node knows about events up to k hops away. It learns which events are likely to be the two ends of one error, and whether the chain joining them crosses the logical operator, which is what flips the label. Its work grows with the number of events, not the size of the code, so it is cheap at low noise. The cost is the bookkeeping of irregular graphs, which PyTorch Geometric handles.

Summary

Model	Tensor into the first layer	PyTorch layer expects	Channels that matter	Empty slots
MLP	`dets [B, 24]`	`nn.Linear: [N, *, in]`	events only	nothing nee empty slots
RNN	`x.reshape(B, 4, 40)`	`nn.GRU: [N, L, H_in]`	events, cum. XOR, mask, final (type folds into the bias)	mask chan `pack_padd` for mixed r
CNN	`[B, 5, 4, 4, 4]`	`nn.Conv3d: [N, C, D, H, W]`	all five	mask chan
Transformer	`x.reshape(B, 32, 5)` → `[B, 32, E]`	`nn.TransformerEncoderLayer: [N, L, E]`	all five, plus slot and round embeddings	`src_key_p` and maske
Graph network	nodes `[N, 4]`, edges `[2, E]`	PyTorch Geometric layers	x, y, t, type per node	nothing nee slots never nodes

Every shape on this page was checked in PyTorch 2.14.1 on the shot shown, including that the RNN's round-1 vector holds X3's five numbers at positions 25–29 and that the Transformer's padding mask hides exactly 8 tokens. The math was checked too: a Linear layer applies one matrix per slot, reordering an MLP's inputs only reorders its weight columns, masked attention satisfies Attn(PU) = P Attn(U), and stacked 3 × 3 convolutions see 3, 5 and 7 positions per side.